Protocoles en Swift

420-249-AH - Programmation mobile iOS

1. Qu'est-ce qu'un protocole ?

En Swift, un protocole définit un contrat qu'un type doit respecter.

Il décrit ce qu'un type doit fournir :

- Propriétés
- Méthodes
- Initialisateurs

mais il ne représente pas directement un objet.

On peut rapprocher le protocole de la notion d'interface connue dans d'autres langages.

2. Déclarer un protocole

Swift utilise :

```
protocol
```

Exemple :

```
protocol Connectable {
    func connecter()
    func deconnecter()
}
```

Le protocole indique : tout type Connectable doit fournir :

```
connecter()
deconnecter()
```

3. Adopter un protocole

Une classe adopte un protocole avec :

```
:
```

Exemple :

```
class Telephone: Connectable {
    func connecter() {
        print("Téléphone connecté")
    }

    func deconnecter() {
        print("Téléphone déconnecté")
    }
}
```

La classe doit respecter le contrat défini par Connectable.

4. Une structure peut aussi adopter un protocole

Les protocoles ne sont pas réservés aux classes.

```
struct Montre: Connectable {
    func connecter() {
        print("Montre connectée")
    }

    func deconnecter() {
        print("Montre déconnectée")
    }
}
```

C'est une différence importante avec l'héritage de classes.

Type	Peut adopter un protocole ?
class	Oui
struct	Oui

5. Propriétés requises

Un protocole peut exiger une propriété.

```
protocol Appareil {
    var nom: String { get }
}
```

get signifie que la propriété doit pouvoir être lue.

Exemple :

```
struct Telephone: Appareil {
    var nom: String
}
```

6. { get set }

Si le protocole exige :

```
var batterie: Int { get set }
```

la propriété doit pouvoir être lue et modifiée.

Exemple :

```swift
protocol Appareil {
    var nom: String { get }
    var batterie: Int { get set }
}
```

7. Implémentation

```swift
class Telephone: Appareil {
    let nom: String
    var batterie: Int

    init(nom: String, batterie: Int) {
        self.nom = nom
        self.batterie = batterie
    }
}
```

Ici :

Propriété	Exigence satisfaite
`nom`	Lecture seulement suffisante
`batterie`	Lecture + modification

8. Méthodes requises

Un protocole déclare la signature de la méthode :

```swift
protocol Rechargeable {
    func recharger()
}
```

Il ne fournit pas nécessairement son fonctionnement. Chaque type décide comment respecter le contrat.

9. Plusieurs implémentations

```swift
class Telephone: Rechargeable {
    func recharger() {
        print("Recharge du téléphone")
    }
}

class Montre: Rechargeable {
    func recharger() {
        print("Recharge de la montre")
    }
}
```

Même contrat — recharger() — mais comportements différents.

10. Plusieurs protocoles

Un type peut adopter plusieurs protocoles.

```
protocol Connectable {
    func connecter()
}

protocol Rechargeable {
    func recharger()
}
```

Puis :

```
class Telephone:
    Connectable,
    Rechargeable {

    func connecter() {
        print("Connexion")
    }

    func recharger() {
        print("Recharge")
    }
}
```

11. Héritage + protocole

Une classe peut hériter d'une autre classe et adopter des protocoles.

```
class Appareil {
}
```

Puis :

```
class Telephone:
    Appareil,
    Connectable,
    Rechargeable {

    func connecter() {
        print("Connexion")
    }

    func recharger() {
        print("Recharge")
    }
}
```

L'ordre est : classe parente, puis protocoles.

12. Héritage vs protocole

L'héritage établit une relation entre classes :

```
Telephone est un Appareil
```

Le protocole exprime plutôt une capacité :

```
Telephone peut être Connectable
Telephone peut être Rechargeable
```

On peut donc penser :

Mécanisme	Répond à
Héritage	Ce que l'objet EST
Protocole	Ce que l'objet PEUT FAIRE

13. Polymorphisme avec un protocole

Un protocole peut également être utilisé comme type commun.

```
protocol Connectable {
    func connecter()
}
```

Deux types différents :

```
class Telephone: Connectable {
    func connecter() {
        print("Téléphone connecté")
    }
}

class Tablette: Connectable {
    func connecter() {
        print("Tablette connectée")
    }
}
```

14. Utiliser any

Dans Swift moderne, lorsqu'on utilise directement un protocole comme type, on peut écrire :

```
any Connectable
```

Exemple :

```
let appareil: any Connectable =
    Telephone()
```

Puis :

```
appareil.connecter()
```

15. Collection de plusieurs types

```
let appareils: [any Connectable] = [
    Telephone(),
    Tablette()
]
```

Puis :

```
for appareil in appareils {
    appareil.connecter()
}
```

Résultat :

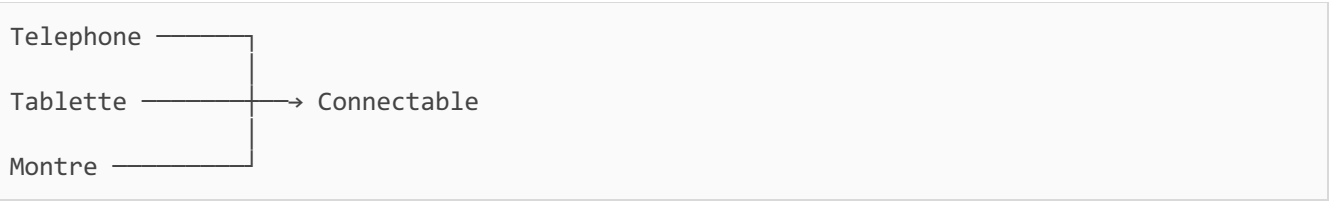
```
Console
Téléphone connecté
Tablette connectée
```

Les objets sont différents, mais ils respectent tous le même contrat.

16. Avantage important

Sans protocole, on pourrait être obligé de créer une hiérarchie artificielle uniquement pour partager un comportement.

Avec un protocole :



Ces types n'ont pas nécessairement besoin d'avoir la même classe parente. Ils doivent simplement respecter Connectable.

17. Protocoles et struct

Une méthode d'un protocole peut devoir modifier une structure. Dans ce cas, le protocole utilise :

```
mutating
```

Exemple :

```
protocol Rechargeable {
    mutating func recharger()
}
```

Puis :

```
struct Batterie: Rechargeable {
    var niveau: Int

    mutating func recharger() {
        niveau = 100
    }
}
```

18. Protocole avec initialisateur

Un protocole peut aussi exiger un initialisateur.

```
protocol IdentifiableAppareil {
    init(nom: String)
}
```

Un type conforme doit fournir cet initialisateur.

Exemple :

```
struct Appareil:
    IdentifiableAppareil {
    var nom: String

    init(nom: String) {
        self.nom = nom
    }
}
```

19. Un protocole peut hériter d'un autre protocole

Exemple :

```
protocol Connectable {
    func connecter()
}
```

Puis :

```
protocol AppareilIntelligent:
    Connectable {
    func synchroniser()
}
```

Un type qui adopte AppareilIntelligent doit donc fournir connecter() et synchroniser().

20. Exemple complet — Téléphone et Montre connectés

```
protocol Connectable {
    var nom: String { get }
    func connecter()
}

protocol Rechargeable {
    var batterie: Int { get set }
    func recharger()
}
```

1. Téléphone

```
class Telephone:
    Connectable,
    Rechargeable {
    var nom: String
    var batterie: Int

    init(
        nom: String,
        batterie: Int
    ) {
        self.nom = nom
        self.batterie = batterie
    }

    func connecter() {
        print("\(nom) connecté")
    }

    func recharger() {
        batterie = 100
        print("\(nom) rechargé")
    }
}
```

2. Montre

```
class Montre:
    Connectable,
    Rechargeable {
    var nom: String
    var batterie: Int

    init(
        nom: String,
        batterie: Int
    ) {
        self.nom = nom
        self.batterie = batterie
    }

    func connecter() {
        print("\(nom) connectée")
    }

    func recharger() {
        batterie = 100
    }
}
```

21. Manipuler les objets par protocole

```
let appareils: [any Connectable] = [
    Telephone(
        nom: "iPhone",
        batterie: 70
    ),
    Montre(
        nom: "Apple Watch",
        batterie: 40
    )
]
```

Puis :

```
for appareil in appareils {
    appareil.connecter()
}
```

Le code dépend du contrat et non d'une classe précise.

22. Protocoles standards de Swift

Swift fournit déjà plusieurs protocoles.

Exemples :

```
Equatable
Comparable
Hashable
Codable
CustomStringConvertible
```

Par exemple, Equatable permet de définir comment deux valeurs peuvent être comparées avec :

```
==
```

Nous rencontrerons plusieurs de ces protocoles dans les applications Swift.

23. Exercice — Appareils connectés

Créer :

```
protocol Connectable
```

avec :

```
func connecter()
func deconnecter()
```

Créer ensuite :

```
class Telephone
struct Montre
class Tablette
```

Les trois types doivent adopter Connectable avec un comportement différent.

Créer ensuite :

```
let appareils: [any Connectable]
```

et appeler connecter() sur tous les appareils avec une boucle.

24. Exercice — Appareils rechargeables

Créer un deuxième protocole :

```
protocol Rechargeable
```

avec :

```
var batterie: Int { get set }
func recharger()
```

Faire adopter ce protocole à certains appareils seulement.

25. À retenir

Un protocole définit un contrat :

```
protocol Connectable {
    func connecter()
}
```

Un type adopte le protocole :

```
class Telephone: Connectable {
}
```

Une classe ou une structure peut adopter plusieurs protocoles.

```
class Telephone:
    Connectable,
    Rechargeable {
}
```

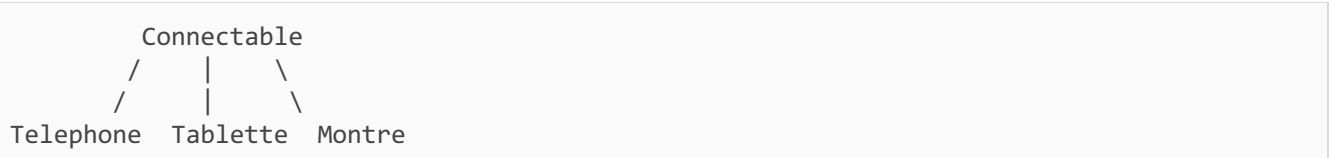
Un protocole peut exiger :

- Propriétés
- Méthodes
- Initialisateurs

et peut servir de type commun :

```
[any Connectable]
```

26. Vision globale



Le protocole permet donc de programmer en fonction d'un contrat, plutôt qu'en fonction d'une classe précise.